COMP101
Introduction to Programming

Pseudocode

Sequence

Problem Statement, Refinement

Problem Specification – from tutorial-03

- Rough Statement supplied by client

“Calculate cost of electricity used”

- Problem Specification after refinement

“For a customer based on their power used as a meter reading given in kilowatt hours (kWh), calculate total electric usage used as the meter reading multiplied by the cost of one kilowatt hour, plus the standing charge and produce a bill showing total electricity used, standing charge and total amount to pay”

Problem Specification into Design

- We have enough detail to design a solution
  - There may be further refinements but we need to be aware some of these are cosmetic and others can be added at code-time
    - Customer name, address etc does not have an impact on the design
  - Don’t add refinements that do not contribute to the fundamental design

- We take the Problem Specification and apply design techniques to it
  - Gives us an algorithm from which we can program a solution
  - Several design techniques exist – ‘top-down design’, ‘stepwise refinement’ etc
    - Structured English, pseudocode, flowcharts, decision tables etc

Pseudocode

- From a ‘Problem Specification’ we can produce a design that is ‘program-language independent’ for coding in any language
  - Avoid statements that belong to specific programming languages
  - Avoid English-like statements that can cause confusion
- Decompose the problem into simpler sub-problems
  - Each sub-problem is easier to solve than trying to solve the whole
  - Sub-problems refine into precise statements using sequence, selection and iteration
- Design techniques are not an exact science – no specified standards
  - Guidelines help us construct them rather than strict rules
  - We identify logic and algorithm statements for translation into a programming language
- It is simpler, easier (and cheaper!) to amend a design than a program

Pseudocode word set and guidelines

Pseudocode is a design technique for reasonably clear and concise design

Does not normally exceed 2 sides of A4 paper and uses a restricted set of words

- set (to initialise variables or constants to a given value)
- input (or read in)
- calculate (can use sum, add, subtract, update etc to describe the calculation)
- print (or write out)
- loop/loopend (avoid ‘loop while’ or ‘loop until’ or ‘loop for’ – too close to coding)
- if – then – else/ifend

Support this with:

- indentation to emphasise structure
- comments for clarification (use # or {})
- decimal numbering of lines

There is no defined standard so there are several variations on these guidelines.
We use it to clarify the logic and structure for programming it.
In our examples here we have included all the major factors you may come across

Problem Specification: draft#1 and #2

refinement

#1 Calculate cost of electricity used

0. Initialise (optional)
    0.1 set cost

1. Input
    1.1 input electric used

2. Process
    2.1 calculate bill

3. Output
    3.1 print(bill)

#2 Take a meter reading to calculate electric usage and produce a bill

0. Initialise (optional)
    0.1 set cost per kWh

1. Input
    1.1 input (meter_reading)

2. Process
    2.1 calculate bill

3. Output
    3.1 print(total)

Problem Specification: draft#3 and #4

#3 For a customer based on their power used as a meter reading given in kilowatthours (kWh), calculate electric usage and produce a bill

0. Initialise

    0.1 set cost per kWh

1. Input

    1.1 input (meter-reading kWh)

2. Process

    2.1 calculate bill from meter reading

3. Output

    3.1 print(total)

#4 For a customer based on their power used as a meter reading given in kilowatt hours (kWh), calculate total electric usage used as the meter reading multiplied by the cost of one kilowatt hour, plus the standing charge and produce a bill

0. Initialise

    0.1 set cost per kWh

    0.2 set standing-charge

1. Input

    1.1 input (meter_reading kWh)

2. Process

    2.1 total = (meter-reading kWh * cost perkWh) + standing-charge

3. Output

    3.1 print(total)

Problem Specification: draft#5 final for coding

#5 For a customer based on their power used as a meter reading given in kilowatt hours (kWh), calculate total electric usage used as the meter reading multiplied by the cost of one kilowatt hour, plus the standing charge and produce a bill showing the total amount due

0. Initialise
    - 0.1 set cost-per-kWh
    - 0.2 set standing-charge
1. Input
    - 1.1 input (meter-reading kWh)
2. Process
    - 2.1 total = (meter-reading kWh * cost-per-kWh) + standing-charge
3. Output
    - 3.1 print(cost per kWh, meter reading (kWh), standing charge, total)

This design is for a sequence construct, hence there are no selection or loop statements.
The designs helps us think about:
0 Initialise:
There may be no need for an initialisation section

1 Input:
Identify what to input (string, numeric etc)

2 Process:
Is the algorithm the best one?

3 Output:
Focus on the main outputs – things like customer name and address can be done in the program itself – these are implementation issues not logic issues

Pseudocode Summary

- For the sample problem this week we have ‘overdone’ the process for the sake of learning
  - Usually no need to go as far as 5 drafts
  - Practice helps identify the key logic aspects earlier on
- Not usual to have sections beginning ‘0. Initialisation’, ‘1. input’ etc
  - A logical approach usually allows us to leave these out

- Avoid trying to program a solution directly from a specification without attention to analysis and design of the program

Pseudocode Summary

- The approach with pseudocode is assume everything will work
  - user input is correct etc.
  - We don't consider validation, testing etc.
- There is no 'right answer'
  - The design helps us identify the logic and structure – which can vary be designer
- Initialising values can be done by user input hence there may be no initialise section
  - If a value remains constant e.g., pi, we could initialise it
- The point is to identify key inputs, processes and outputs to a realistic level of detail to construct a working program
  - Makes us think about the constructs, the logic and what data types and structures may be needed

Pseudocode Summary

Does not exceed 2 sides of A4 paper using a restricted set of words

- set to initialise variables or constants to a given value
- input (or read in) – what is user expected to type in
- calculate can use sum, add, subtract, update etc for calculation
- print (or write out) usually to screen but implies printer or file
- loop/loopend avoid ‘loop while’ or ‘loop until’ or ‘loop for’
- if – then – else/ifend don’t use ‘elif’ etc – specific to Python

Avoid using anything that associates with a specific programming language